

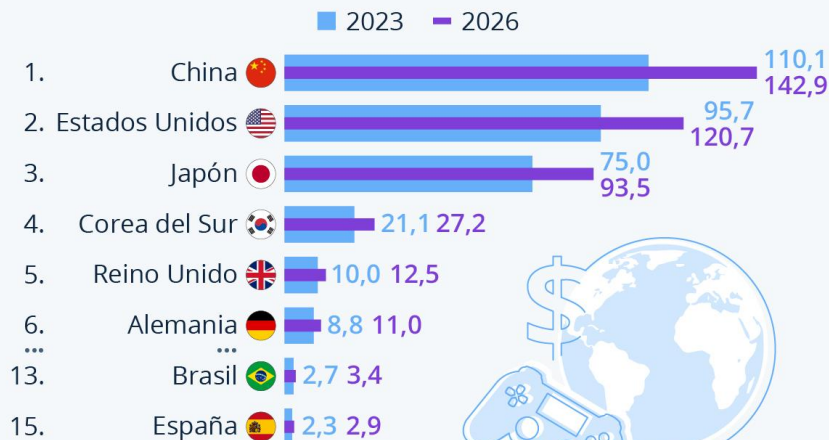
Index

- | | | | |
|----------|--|----------|---------------------|
| 1 | Indústria del videojoc | 5 | Metodologies |
| 2 | Departaments a un estudi de videojocs | 6 | Conceptes |
| 3 | Procés de creació d'un videojoc | 7 | Git |
| 4 | Eines per la gestió del projecte | 8 | Repte |

Industria del videojoc

Los principales mercados de videojuegos

Países con los mayores ingresos generados por la industria de los videojuegos (en miles de millones de USD)



Estimaciones de agosto de 2023.

Fuente: Statista Market Insights



statista

es.statista.com



Infografía: Los mercados de videojuegos más grandes del mundo

Este gráfico muestra los seis principales mercados de los videojuegos en el mundo, así como los ingresos de esta industria en Brasil y España.



Los videojuegos generarán **US\$187.700 millones** en 2023, un crecimiento interanual de **2,6%**



El mercado indie en 2020, alcanzó un valor de **US\$2.000 millones**, y en 2021, representaron **40%** de todos los productos vendidos en Steam, uno de los principales distribuidores de videojuegos

1

Elegir el motor adecuado: Unity y Unreal son los más populares, y tienen versiones gratuitas



Programas para el diseño de audio y sonido



Según el motor deberá aprender un lenguaje de programación (C#, C++, JavaScript, Python o Lua)



Diseño de los niveles y la mecánica del juego



Pruebas y depuración: crucial para identificar y solucionar los problemas del juego



Usar programas para los gráficos visuales (modelado 3D: Maya, Blender, 3D Max; para la edición gráfica Photoshop)



Publicación y distribución

Departaments a un estudi de videojocs

Client



- Qui ens encarrega
- compra
- publica el joc

Desenvolupament



- Programadors
- Artistes
- Game Designers

Operacions



- Màrqueting
- Producció
- Monetització

Departaments a un estudi de videojocs (Desenvolupament)



Programming	Art	System Design	Audio	Management
Software developers Mobile developers Virtual / Augmented Reality developers Artificial intelligence engineers Graphics engineers Network engineers Gameplay programmers Quality assurance technicians	3D artists 2D artists Lighting artists Character artists Technical artists Visual Effects artists Animators Cinematographers User Interface (UI) Designer	Game Designers Level Designers Systems architect Script writer Narrative designer User Experience (UX) Designer	Audio designer Sound engineer Composer	Producers Product owners Creative directors QA managers

Departaments a un estudi de videojocs (Operacions)

El departament d'operacions en una empresa de videojocs té un paper crucial per assegurar que els jocs es desenvolupin, distribueixin i mantinguin de manera eficient. A continuació, et mencionaré alguns dels especialistes i les seves funcions dins d'aquest departament:

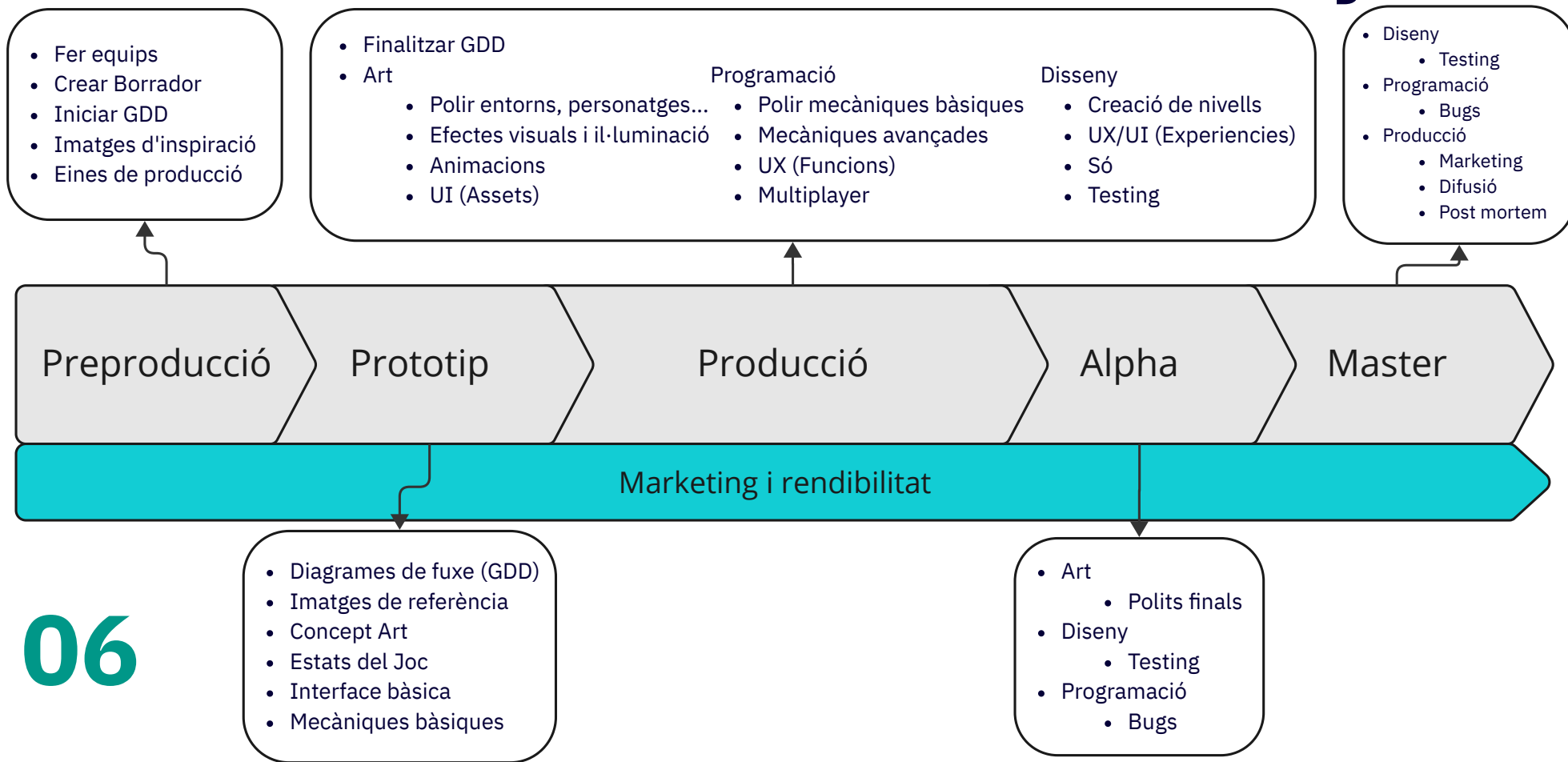


05

Gerent d'Operacions

Supervisa tot el departament d'operacions i s'encarrega de la planificació estratègica, la gestió de recursos i la presa de decisions clau per assegurar que el procés de desenvolupament de videojocs es porti a terme de manera efectiva i eficient.

Procés de creació d'un videojoc



Eines per la gestió del projecte



Metodologies

Waterfall

- Les tasques van una darrere l'altra.
- Diagrama de Gantt.



Metodologies

Agile, Scrum

- Kanban.
- Concurrència.
- Sprints.
- Iteracions.
- Agilitat, facilitat per respondre a canvis en les funcionalitats o els requeriments del projecte.

Scrum

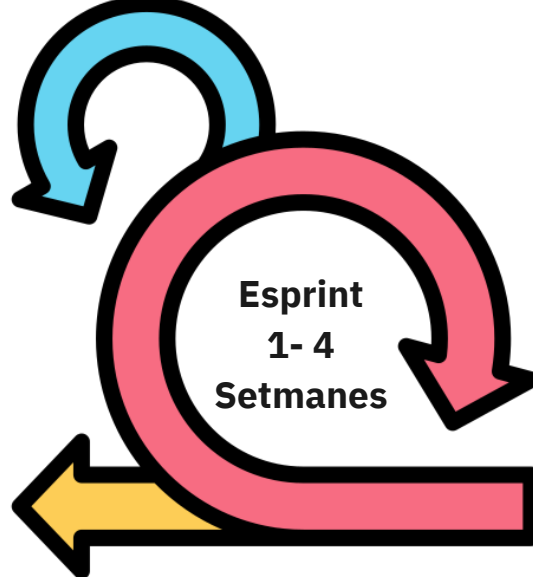


Reunió diària de sincronització (15 m)

Increment de producte



- revisió de sprint (1,5 h)
- retrospectiva de l'esprint (1,5 h)

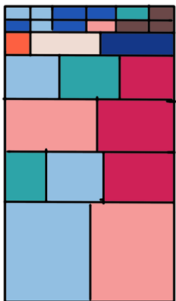


Client

aportacions d'executius,
temes, grups d'interès,
clients i usuaris



Product Owner



Product Backlog

Llista classificada
del que es
requereix:
característiques,
històries d'usuari



Equip



Scrum Master

L'equip selecciona començant per
dalt tant com es pugui
comprometre a lliurar al final de
l'esprint

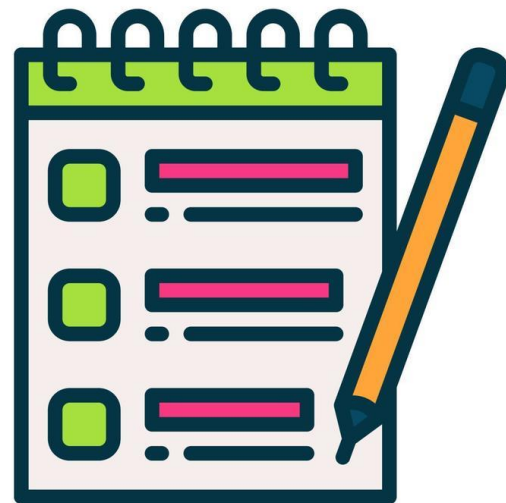
- Selecció de requisits (2 hores).
- Planificació de la iteració (2 hores).

Reunió de planificació de l'esprint

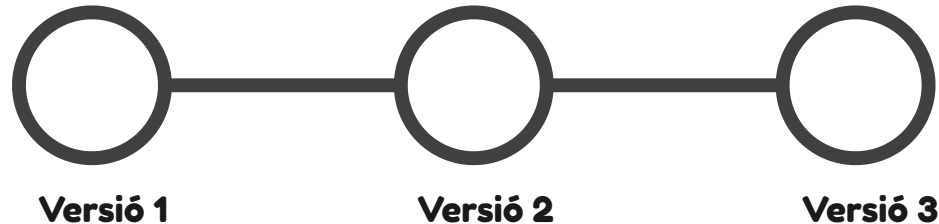
Tasca (Issue, User Story, ...)

Per cada tasca

- Nom
- Que se'n fa
- Assignació
- Data de venciment
- Estat:
 - Backlog
 - To Do
 - In progress
 - QA Testing
 - Complete (Done)
- Revisor
- Quan de temps ha tardat
- Commit del github i/o branca
- Que s'ha fet i que s'ha de testear
- Documentació de com testear
- Assignar subtasques necessàries.



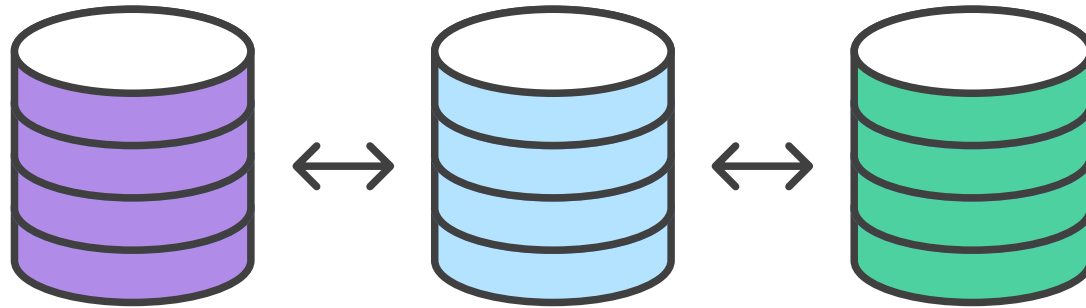
Control de versions (Git)



Mantenir un sistema de control de versions és com dur un diari dels canvis realitzats damunt l'aplicació.

A més tenint en compte que hi ha les versions de cada un dels desenvolupadors que participen en el projecte.

Git té estructura distribuïda



A diferència de molts altres mecanismes de control de versions, com CVS o Subversion, **Git no necessita un repositori central definitiu**. En lloc d'això, tots els repositoris tenen el mateix estat i qualsevol d'ells pot actualitzar-se amb la resta. És una cosa que funciona molt bé en projectes altament cooperatius.

Conceptes de Git

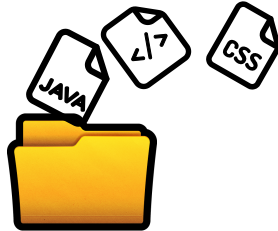
REPOSITORI

Base de dades on s'amagatzema l'historial del codi.



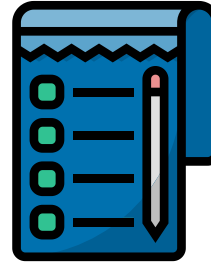
STAGE

Llista d'arxius que s'inclouran al commit.



COMMIT

Registre d'un o varis canvis fets en el repositori



FORK

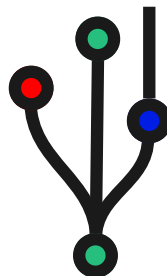
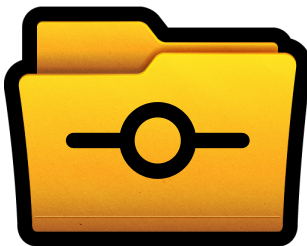
Copia d'un repositori



Conceptes de Git

MASTER

Rama principal
d'un repositori

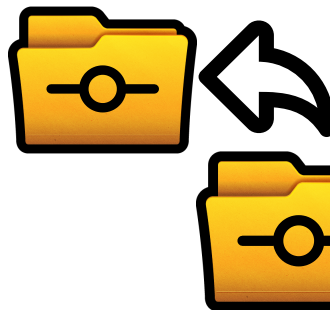


BRANCH / RAMA

Entorn o lloc de feina
independent dins GIT

MERGE

Combinam dues
o més rames
amb una



CHECKOUT

Mourer-se entre
diferents rames.

Comandes de Git

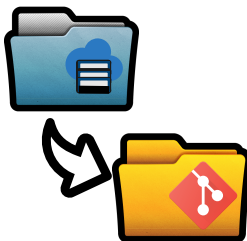
```
>_ git init
```

Inicia un repositori en local



```
>_ git clone
```

Clona un repositori existent



```
>_ git reset
```

move el projecte a un commit anterior eliminant tots els posteriors de l'història de commits.



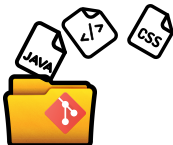
```
>_ git status
```

Enumera tot els arxius que han estat modificats i necessiten fer commit.



```
>_ git add
```

Afegeix un o més arxius a la base de dades del repositori



```
>_ git branch
```

Mostra la llista de branques del repositori



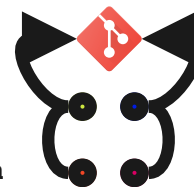
```
>_ git help
```

Mostra informació per conèixer l'ús complet de cada comanda de git



```
>_ git diff
```

Mostra les diferències dels arxius que encara no s'ha fet commit.



Comandes de Git

```
>_ git log
```

Ens llista l'història de versions de la rama actual



```
>_ git config
```

Mostra la llista dels paràmetres que Git utilitza per defecte.



```
>_ git clean
```

Elimina tots els arxius que no estan inclosos dins el control de versions.



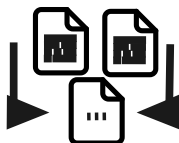
```
>_ git pull
```

Descarrega i actualitza els canvis realitzats al repositori remot al repositori local.



```
>_ git merge
```

Fusiona dues o més rames dins una.



```
>_ git push
```

Pujar els arxius al repositori remot.



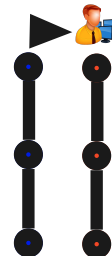
```
>_ git remote
```

Crear, visualitzar i eliminar connexions amb altres repositoris



```
>_ git checkout
```

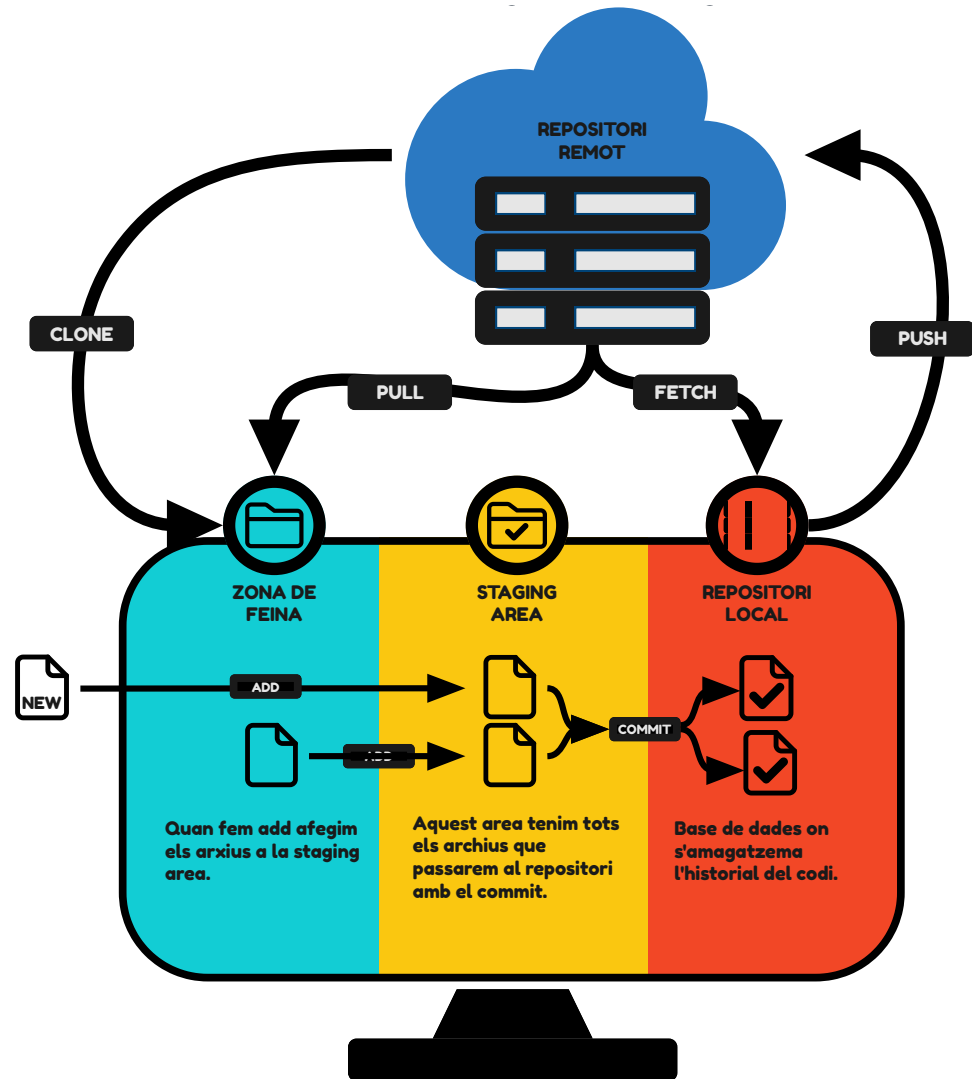
Serveix per moure-se d'una rama a un altre i moure-te amb el temps.



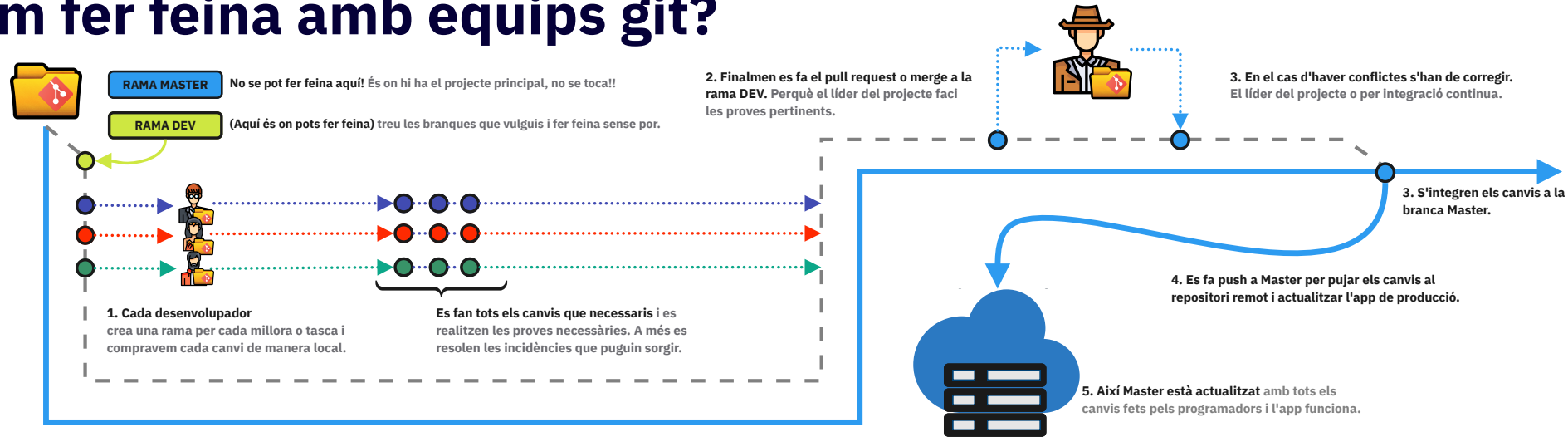
Flux de treball local/remot

En Git, hi ha poques comandes que interactuen amb un repositori remot.

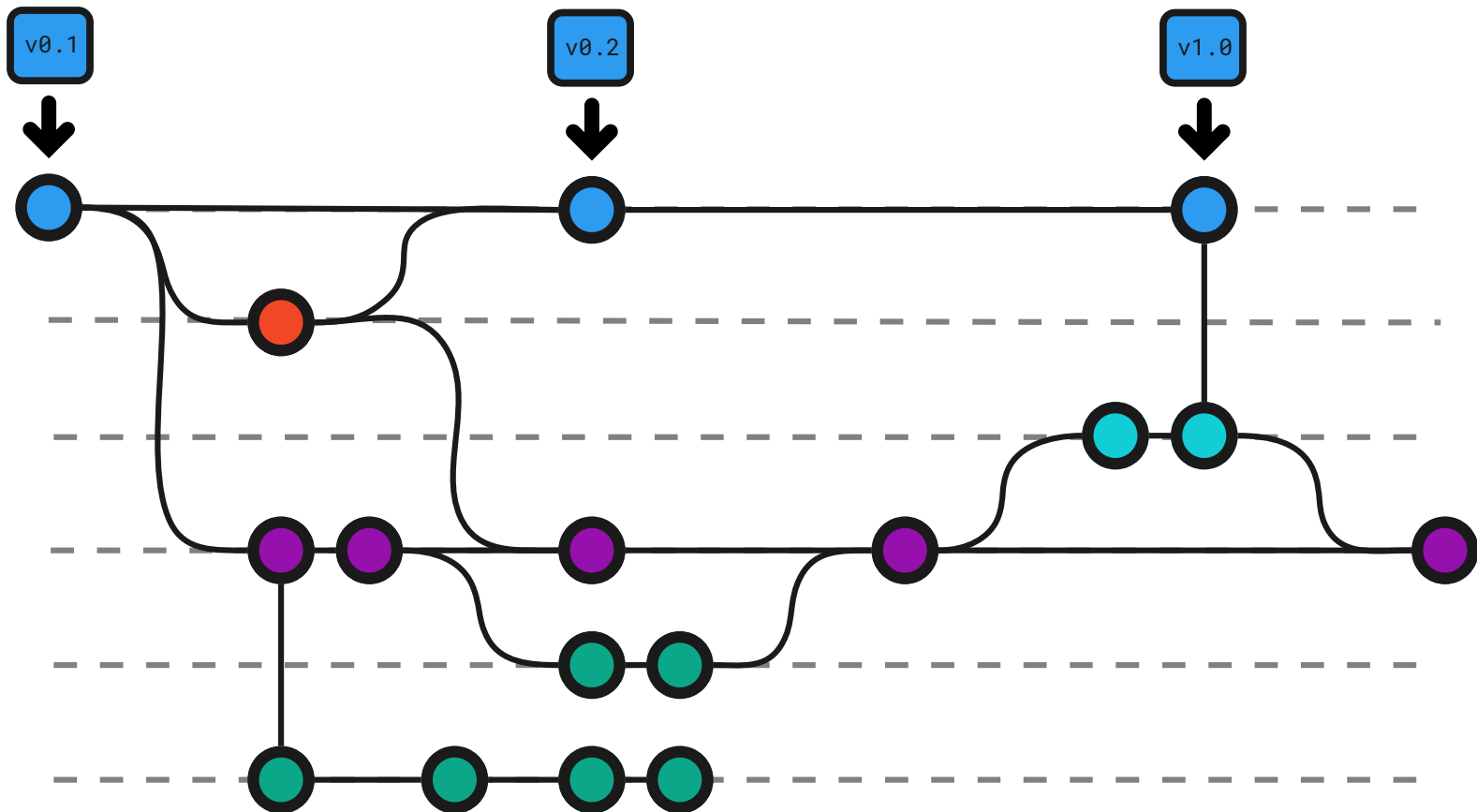
La immensa majoria de la feina es fa en el repositori local. Fins a aquest punt (excepte quan diem "git clone"), hem treballat exclusivament amb el nostre repositori local de Git i mai hem sortit del nostre ordinador local.



Com fer feina amb equips git?



Flux de treball el GITFLOW



Flux de treball el GITFLOW

Master

La màster emmagatzema l'historial de versions oficial. És convenient etiquetar (git tag) els commits a master amb un nombre de versió.

Develop

la develop serveix com una sucursal d'integració per a les funcions.

Feature

Cada nova característica ha de residir en la seva pròpia branca, que pot enviar-se al repositori central per suport/col·laboració. Però, en lloc de crear un rama de màster, les features utilitzen develop com la seva branca principal.

Release

Una vegada que develop ha adquirit suficients característiques per a un llançament, desemborsar una release fora de develop. La creació d'aquesta branca inicia el següent cicle de llançament, de manera que no es poden afegir noves característiques després d'aquest punt.

hotfix

El manteniment o les rames "hotfix" s'utilitzen per posar parches ràpidament la versió de producció.

Missatges idals pels commits

```
>_ git commit
```

Primer un títol amb un resum breu de menys de 50 caràcters amb els canvis realitzats.

Explicació molt més detallada que dona context i aporta informació addicional. Una bona pràctica és fer una llista d'oracions enumerant els canvis realitzats.

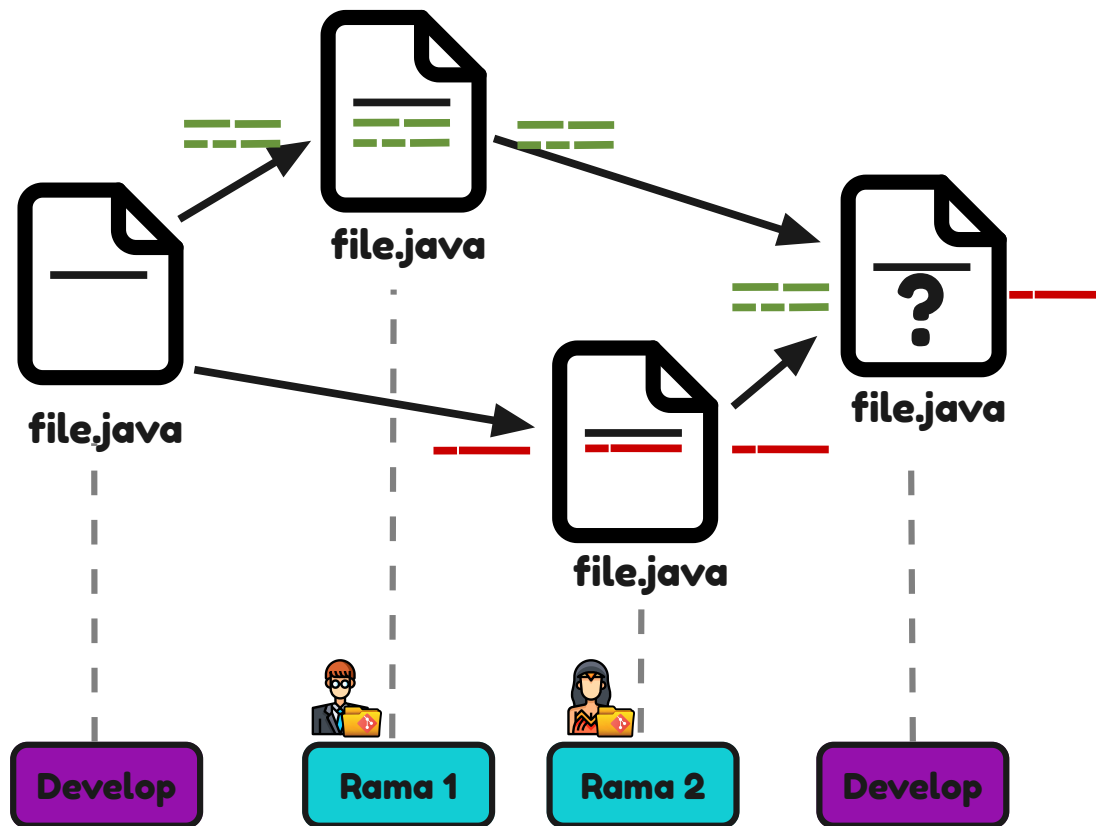
Finalment indicam l'identificador de les issues o tasques relacionades.

Missatges idals pels commits

Recordatori del git commit

La comanda "git commit" captura una instantània dels canvis realitzats actualment en el projecte. Les instantànies confirmades poden considerar versions "segures" d'un projecte; Git mai les canviarà a menys que vostè ho demani explícitament. Abans de l'execució de git commit, s'ha de fer el git add comanda que s'utilitza per promoure o "preparar" canvis en el projecte que s'emmagatzemaran en una confirmació. Aquestes dues ordres git commit i git add són dues de les més utilitzades.

Conflictes que són?

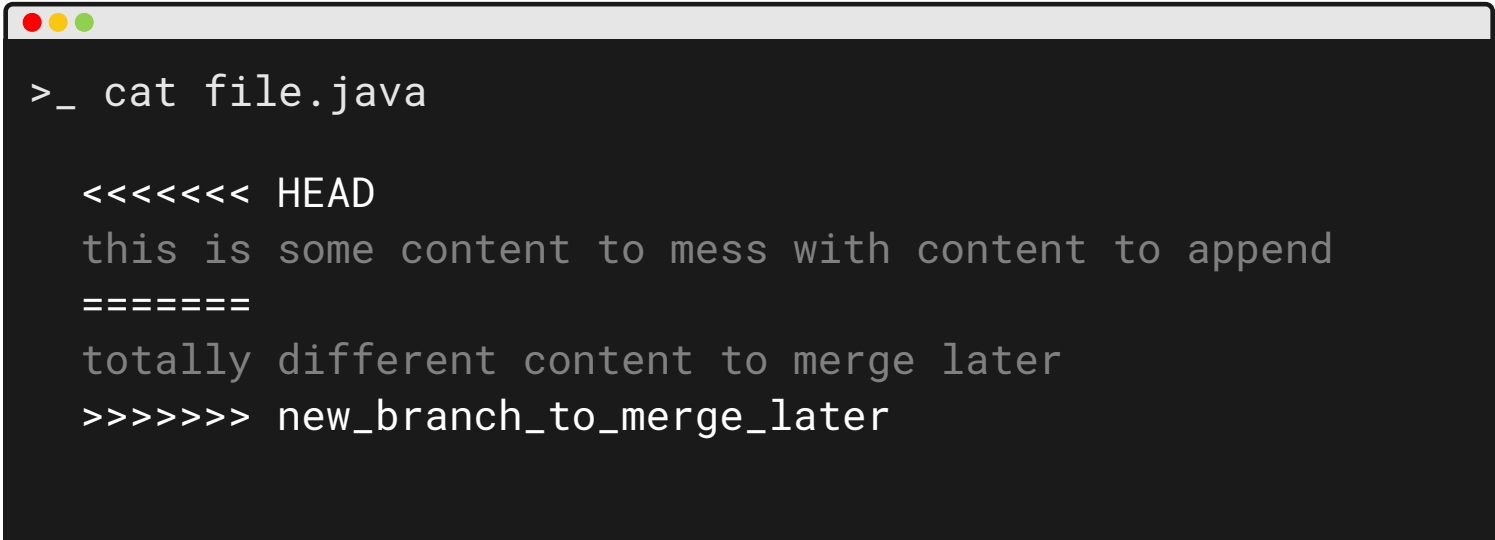


Conflictes que són?

Els conflictes generalment sorgeixen quan dues persones han canviat les mateixes línies en un arxiu, o si un desenvolupador va esborrar un arxiu mentre un altre desenvolupador ho estava modificant. En aquests casos, Git no pot determinar automàticament què és correcte. Els conflictes sol afecten el desenvolupador que realitza la fusió, la resta de l'equip desconeix el conflicte. Git marcarà l'arxiu com conflictiu i aturarà el procés de fusió. Llavors, és responsabilitat dels desenvolupadors resoldre el conflicte.

Conflictes que són?

Com es poden veure els conflictes



```
>_ cat file.java

<<<<<< HEAD
this is some content to mess with content to append
=====
totally different content to merge later
>>>>>> new_branch_to_merge_later
```

Repte (Disseny d'una Pàgina Web de Venda de Videojoc)

Objectiu del Projecte:

Crear una pàgina web atractiva, funcional i persuasiva per a potencials compradors.

Requisits del Projecte:

Selecció del Videojoc: Cada grup d'estudiants ha de seleccionar un videojoc fictici que vulguin vendre a la pàgina web. Han de descriure les característiques i la història del joc de manera detallada.

Disseny Visual: Els estudiants han de dissenyar una interfície d'usuari atractiva i coherent per a la pàgina web. Això inclou la creació de logotips, icones i gràfics relacionats amb el videojoc.

Contingut Multimèdia: Els estudiants han d'incorporar imatges, vídeos o animacions relacionats amb el videojoc per augmentar l'atractiu visual de la pàgina.

Informació de contacte: s'ha de proporcionar informació de contacte clara perquè els usuaris puguin fer preguntes o resoldre problemes relacionats amb la compra.

Pàgines Principals: La pàgina web ha de contenir almenys les següents pàgines principals:

1. Pàgina d'inici (amb informació sobre el joc i una crida a l'acció per comprar-lo).
2. Pàgina de detalls del joc (amb imatges, descripció, requisits del sistema, etc.).
3. Pàgina de compra (on els usuaris poden seleccionar la quantitat i fer la compra).
4. Pàgina de contacte (amb informació de contacte i un formulari per a consultes).

Funcionalitat de Compra: S'ha d'implementar un sistema de carret de compres que permeti als usuaris seleccionar la quantitat de còpies que volen comprar i fer la compra de manera segura.

Disseny Responsiu: La pàgina web ha de ser completament funcional en dispositius mòbils i tauletes, a més d'ordinadors d'escriptori.

Documentació: Els estudiants han de lliurar una documentació detallada que expliqui les decisions de disseny, les tecnologies utilitzades i els desafiaments superats durant el procés de desenvolupament.